Midterm 2

Put your name here

DATA 412/612

2026-11-11

November 11, 2026. You have 75 minutes. Questions 1 to 4 are 100 points, 25 each. Question 5 is 10 points of extra credit.

Put your name in the subtitle at the top of this file. Write each answer in the chunk under that part. Render to PDF. Submit this `.qmd` and the PDF before the 75 minutes are up.

- You may use your notes and the R help files inside RStudio.
- The only website you may use is https://amousavi1.github.io/612/data-612.html.
- You may not use Google, any other website, or an LLM.
- You may not communicate with anyone else.
- Use the tidyverse where a tidyverse verb exists. For text, use stringr. Question 5 is a function; base R is appropriate for the `stop()` check.
- The setup chunk loads tidyverse, DBI, RSQLite, and dbplyr. The database is the European soccer file from the database notes, `soccer.sqlite`. Put it at `data/soccer.sqlite`, in a folder `data` next to this file. Connect with that relative path.
- `collect()` before `separate()`, `unite()`, or any `str_` verb. Those run in R. This SQLite file does not run a regular expression.
- Either pipe is fine: `|>` or `%>%`.
- Partial credit is given for code that runs. If you are stuck, go to the next part. Later questions may use the connection `con` from question 1. Open the database again if you do not have it.

1. The database (25 points)

(a) Connect and list the tables (8 points)

Connect to `data/soccer.sqlite` with `dbConnect()` and `SQLite()`. Assign the connection to `con`.

Print the table names. Do not count `sqlite_sequence`. How many tables remain? Print that number.

(b) A key (9 points)

Using `tbl()`, report two numbers for `Player` and the same two numbers for `Player_Attributes`: the number of rows, and the number of distinct `player_api_id`.

In one sentence, say whether `player_api_id` is a primary key of each table, and why.

(c) One season (8 points)

From `Match`, keep season `"2015/2016"`. Keep `date`, `home_team_goal`, and `away_team_goal`. Add `goal_diff`, equal to `home_team_goal - away_team_goal`.

Show the SQL with `show_query()`. Then `collect()`. Print the number of matches and the mean of `goal_diff`.

In one sentence, say what `collect()` changed about the object.

2. Player names (25 points)

`collect()` `player_api_id` and `player_name` from `Player`. The rest of this question is ordinary R.

(a) Simple names (10 points)

A name is simple when `player_name` has exactly two uppercase letters and exactly one space. Use `str_count()` for both.

Keep the simple names. Split `player_name` into `first_name` and `family_name` on the space, with `separate()`.

How many simple names are there? Print that number. Then print the first six rows, arranged by `family_name` and, within that, by `first_name`. Keep `player_api_id`, `first_name`, and `family_name`.

(b) Two ways to keep a player (8 points)

From the simple names, keep a row when either of these is true:

- `first_name` is `"Luis"` or `"Kevin"`
- `family_name` ends in `ez`

Use `str_ends()` for the ending.

How many rows remain? Among those rows, show `first_name` and `family_name` for every player whose family name is `"Martinez"`, arranged by `first_name`.

(c) Initials (7 points)

On the simple names from part (a), build `initials` with `str_sub()` and `str_c()`. Take the first letter of `first_name`, a period, the first letter of `family_name`, and a period. `Luis Martinez` becomes `L.M.`.

Which five initials occur most often? Show the initial and its count, largest count first. Break a tie by the initial, A to Z.

3. Potential and sprint speed (25 points)

The plot in part (c) is the same kind of picture as the database sample: mean potential against the standard deviation of sprint speed. The players are a different set.

You may reuse the simple names from question 2. Build them again if you do not have them. A simple name has exactly two uppercase letters and exactly one space, then `first_name` and `family_name`.

(a) The players (8 points)

From the simple names, keep rows whose `first_name` starts with `M` and whose `family_name` has at least 8 characters. Use `str_starts()` and `str_length()`.

Put `first_name` and `family_name` back together into `player_name`, with one space between them, using `unite()`. Keep `player_api_id` and `player_name` only. Call the result `named`.

How many rows does `named` have?

(b) One row per player (9 points)

From `Player_Attributes`, keep `player_api_id`, `potential`, and `sprint_speed`.

For each `player_api_id`, report:

- `potential_avg`, the mean of `potential`
- `sd_speed`, the standard deviation of `sprint_speed`

Ignore missing values. You may summarise in the database and then `collect()`, or `collect()` the three columns and summarise in R.

(c) Join and plot (8 points)

Left join `named` to the summary from part (b) by `player_api_id`. Drop a row when `potential_avg` or `sd_speed` is missing.

Plot `sd_speed` on the horizontal axis and `potential_avg` on the vertical axis.

- Points, with an alpha so overlapping points stay visible.
- One straight smooth in black, dashed, with no gray band.
- Use `theme_bw()`. Title the plot `Potential and sprint speed`. Label the axes `SD of sprint speed` and `Mean potential`.

In one sentence, say how mean potential changes as the standard deviation of sprint speed increases.

4. Team names and birthdays (25 points)

(a) Long names (10 points)

`collect()` `team_api_id`, `team_long_name`, and `team_short_name` from `Team`.

A name has a non-ASCII character when `str_detect()` matches the pattern `[^\\x00-\\x7F]`. That pattern is one character outside the basic ASCII set. Accented letters match. How many long names match? Print that count.

Build `team_name` by removing a leading run of digits, a period, and the space after that period. `1. FC Köln` becomes `FC Köln`. Use one `str_remove()`, with the pattern `^\\d+\\.\\s+`.

Show `team_long_name` and `team_name` for the rows that change.

(b) Short names and FC (8 points)

Use `team_name` from part (a).

Which teams have a `team_short_name` whose length is not 3? Use `str_length()`. Show `team_name` and `team_short_name`.

How many values of `team_name` contain `FC` as a whole word? Use `str_detect()` with `\\bFC\\b`.

How many contain the letters `FC` anywhere? Show the names that this second count includes and the whole-word count does not.

In one sentence, say why those names are in one count and not the other.

(c) The month in the birthday (7 points)

`birthday` in `Player` is text, such as `1992-02-29 00:00:00`. `collect()` that column.

With one `str_match()`, take the year, the month, and the day. The pattern is `^(\\d{4})-(\\d{2})-(\\d{2})`. In the matrix this returns, column 1 is the whole match, column 2 is the year, column 3 is the month, and column 4 is the day.

How many players have month `02`? How many have month `02` and day `29`?

In one sentence, say why February 29 can still be a real birthday in this table.

5. A function (10 extra credit)

Write a function of one argument, `name`.

- If `name` is not a character vector of length 1, or it is missing, stop with the message `name must be one string`.
- Collapse the whitespace with `str_squish()`.
- If a comma appears, drop the comma and everything after it with `str_remove()`, then squish again.
- If the result contains no space, return it unchanged.
- Otherwise return the last word, with `str_extract()` and the pattern `\\S+$`.

Write the function in this chunk, then run it on these four inputs. This chunk already has `#| error: true`, so the last call can stop and the document still renders. Leave that error visible.

- `"  Luis   Martinez "`
- `"Abdoulaye Diallo Sadio,22"`

- "Luisinho"
- c("Aaron", "Taylor")